

RAPPORT TECHNIQUE

Projet CI/CD Pipeline Sécurisé

Explication complète du fonctionnement

Flask · Jenkins · Docker · Harbor · Trivy · Bandit · Gitleaks · Ansible · Kubernetes

Auteur	Mara Sylvain LAHINIRIKO
Date	26 Février 2026
Environnement	Parrot OS · Docker 29.2.1 · Minikube v1.38.1 · Jenkins LTS 2.528.3
Stack	Jenkins · GitHub · Docker · Harbor v2.10.0 · Trivy v0.69.1 · Bandit v1.9.3 · Gitleaks v8.18.4 · Ansible ·
Application	Flask 3.0.3 · Python 3.12 Alpine · namespace: production · 2 replicas

- Jenkins
- GitHub
- Docker
- Harbor
- Trivy
- Bandit
- Gitleaks
- Ansible
- Kubernetes



Table des matières

1. Vue d'ensemble — Architecture générale et flux du pipeline
2. Le Jenkinsfile — Cerveau du pipeline
3. Ansible — Gestion de configuration
4. Les rôles Ansible — Harbor, Minikube, Jenkins, Kubernetes
5. Harbor Registry — La registry Docker privée
6. Les outils de sécurité — Trivy, Bandit, Gitleaks
7. Le déploiement Kubernetes — Manifest et rolling update
8. Flux d'exécution complet — Du git push aux pods Running

1 Vue d'ensemble — Architecture générale et flux du pipeline

Ce projet met en place un **pipeline CI/CD complet et sécurisé** pour une application web Python (Flask). L'objectif est d'automatiser entièrement le cycle de vie de l'application : depuis l'écriture du code par le développeur jusqu'à son déploiement sur un cluster Kubernetes, en intégrant des contrôles de sécurité à chaque étape. Le pipeline suit l'approche **DevSecOps** — la sécurité n'est pas une phase finale, elle est intégrée dans chaque maillon de la chaîne.

Dès qu'un développeur pousse du code sur GitHub, un **WebHook** déclenche automatiquement Jenkins. Jenkins orchestre alors l'ensemble des opérations : analyse de sécurité du code, construction de l'image Docker, scan de vulnérabilités, publication dans Harbor, et déploiement sur Kubernetes via Ansible. Aucune intervention manuelle n'est nécessaire.

1.1 Les composants de la stack

Composant	Rôle	Version
Jenkins (Docker)	Orchestrateur CI/CD — lit le Jenkinsfile et coordonne toutes les étapes	LTS 2.528.3
GitHub	Dépôt de code source et déclencheur de pipeline via WebHook	—
Docker	Construction et publication des images conteneur	29.2.1
Harbor	Registry Docker privée locale (remplace Docker Hub)	v2.10.0
Trivy	Scanner CVE des images Docker — bloque si CRITICAL/HIGH	v0.69.1
Bandit	Analyseur statique Python (SAST) — vulnérabilités code source	v1.9.3
Gitleaks	Détecteur de secrets exposés dans l'historique Git	v8.18.4
Ansible	Gestionnaire de configuration — infrastructure as code	2.x
Minikube	Cluster Kubernetes local mono-noeud	v1.38.1
Kubernetes	Orchestration des conteneurs applicatifs en production	v1.35.1
Flask (Alpine)	Application web Python déployée — image 54.7 MB	3.0.3

1.2 Flux du pipeline — Vue globale

```
git push

|

GitHub WebHook

|

Jenkins (orchestre tout)
```

| | |

Bandit/Trivy Docker Build Gitleaks

(sécurité code) (image Alpine) (secrets)

|

Harbor Registry

(harbor.local/cicd/flask-cicd-app:N)

|

Ansible Playbook

(ansible-playbook deploy.yml)

|

Kubernetes (Minikube)

2 pods Running – namespace: production

2 Le Jenkinsfile — Cerveau du pipeline

Le **Jenkinsfile** est le fichier central du système. Il est écrit en Groovy et versionné directement dans le dépôt GitHub avec le code source. Jenkins le lit et exécute les étapes dans l'ordre. Ce fichier implémente le paradigme **Pipeline as Code** : l'infrastructure de déploiement est traitée comme du code source, avec tous les avantages associés — historique des modifications, revue de code, possibilité de rollback.

2.1 Structure générale

Le fichier s'organise en quatre blocs. Le bloc **environment** centralise toutes les variables (adresse Harbor, nom du projet, tag de l'image). Le bloc **options** définit un timeout global de 30 minutes. Le bloc **stages** contient les neuf étapes séquentielles. Le bloc **post** exécute les actions de nettoyage (déconnexion Harbor) que le pipeline réussisse ou échoue.

```
pipeline {  
  
    agent any // Tourne sur n'importe quel agent Jenkins  
  
    environment { } // Variables disponibles dans tout le pipeline  
  
    options { } // Comportement global (timeout, etc.)  
  
    stages { } // Les étapes dans l'ordre  
  
    post { } // Actions après le pipeline (succès/échec)  
  
}
```

2.2 Le bloc environment — Variables du pipeline

Toutes les variables sont définies au même endroit pour éviter la duplication. La ligne **credentials('harbor-credentials')** est particulièrement importante : elle dit à Jenkins d'aller chercher le login/password Harbor dans son coffre-fort sécurisé. Jenkins crée alors automatiquement deux variables d'environnement : **\$HARBOR_CREDENTIALS_USR** pour le nom d'utilisateur, et **\$HARBOR_CREDENTIALS_PSW** pour le mot de passe — ce dernier est automatiquement masqué dans tous les logs.

```
environment {  
  
    HARBOR_URL = "harbor.local" // Adresse de la registry privée  
  
    HARBOR_PROJECT = "cicd" // Projet dans Harbor  
  
    HARBOR_CREDENTIALS = credentials('harbor-credentials')  
  
    // credentials() crée automatiquement :
```

```
// $HARBOR_CREDENTIALS_USR → le username

// $HARBOR_CREDENTIALS_PSW → le password (masqué dans les logs)

IMAGE_NAME = "${HARBOR_URL}/${HARBOR_PROJECT}/flask-cicd-app"

IMAGE_TAG = "${BUILD_NUMBER}" // Tag = numéro du build (1, 2, 3...)

K8S_NAMESPACE = "production"

APP_NAME = "flask-app"

}
```

2.3 Les neuf étapes du pipeline

Chaque étape a un rôle précis. Si l'une d'elles échoue, le pipeline s'arrête immédiatement et empêche la propagation du problème aux étapes suivantes.

Étape 1 — Clone Repository

Récupère le code depuis GitHub. La directive **checkout scm** utilise automatiquement l'URL du dépôt configurée dans le job Jenkins. C'est le point de départ de chaque build.

Étape 2 — Bandit (SAST)

Analyse statique du code Python. Bandit cherche des failles de sécurité sans exécuter le code : injections SQL, mots de passe en clair, appels système dangereux. L'option **--severity-level medium** signifie qu'il remonte uniquement les problèmes de niveau moyen ou supérieur, évitant le bruit des faux positifs mineurs.

Étape 3 — Docker Build

Construit l'image Docker de l'application Flask avec le tag correspondant au numéro du build Jenkins. L'option **--no-cache** force une reconstruction complète à chaque fois pour éviter d'utiliser des couches d'image périmées qui pourraient contenir des dépendances obsolètes.

Étape 4 — Trivy (scan local)

Scanne l'image Docker construite à la recherche de CVE (Common Vulnerabilities and Exposures). L'option **--exit-code 1** fait échouer le pipeline si une CVE de niveau CRITICAL ou HIGH est trouvée. C'est le premier gate de sécurité — une image vulnérable ne sera jamais publiée dans Harbor.

Étape 5 — Gitleaks

Analyse l'intégralité de l'historique Git à la recherche de secrets accidentellement commités : clés API, mots de passe, tokens d'authentification. Cette étape protège contre une fuite de credentials dans le code source.

Étape 6 — Harbor Push

Publie l'image dans Harbor. Le mot de passe est transmis via stdin avec **--password-stdin** pour éviter qu'il n'apparaisse dans les logs Jenkins ou dans l'historique bash. Deux tags sont poussés : le tag versionné (:BUILD_NUMBER) et le tag :latest.

```
echo $HARBOR_CREDENTIALS_PSW | docker login ${HARBOR_URL} \  
  
-u $HARBOR_CREDENTIALS_USR --password-stdin  
  
docker push ${IMAGE_NAME}:${IMAGE_TAG} // Ex: harbor.local/cicd/flask-cicd-app:42  
  
docker push ${IMAGE_NAME}:latest
```

Étape 7 — Trivy Remote

Re-scanner l'image directement depuis Harbor (pas depuis le cache local Docker) pour confirmer que ce qui est stocké dans la registry est bien propre. L'option **--exit-code 0** ici — cette étape est informative seulement et ne bloque pas le pipeline.

Étape 8 — Ansible Deploy

C'est la nouveauté par rapport à l'architecture précédente. Au lieu d'exécuter *kubectl apply* directement, Jenkins appelle Ansible qui gère l'ensemble du déploiement de façon déclarative. La variable **image_tag** est passée en paramètre pour déployer exactement la version correspondant au build courant.

```
ansible-playbook \  
  
-i ansible/inventory/hosts.ini \  
  
ansible/deploy.yml \  
  
-e "image_tag=${IMAGE_TAG}" # passe le tag en variable
```

Étape 9 — Vérification finale

Exécute des commandes *kubectl get pods*, *kubectl get services* et *kubectl get events* pour confirmer visuellement dans les logs Jenkins que les 2 pods Flask sont bien en état Running dans le namespace production.

3 Ansible — Gestion de configuration

Ansible est l'outil qui remplace **tous les scripts bash manuels** dans ce projet. Son principe fondamental est l'**idempotence** : une même opération peut être exécutée plusieurs fois sans produire d'effet indésirable. Si l'état désiré est déjà atteint, Ansible ne fait rien. Cela permet de relancer les playbooks après un redémarrage ou une interruption sans risquer de casser quelque chose.

Un autre avantage majeur est la **traçabilité** : toute la configuration de l'infrastructure est versionnée dans Git sous forme de fichiers YAML lisibles. N'importe qui peut comprendre ce que fait le système en lisant les playbooks, et recréer l'environnement entier depuis zéro avec une seule commande.

3.1 L'inventaire — `ansible/inventory/hosts.ini`

L'inventaire Ansible liste les machines sur lesquelles les playbooks s'exécutent. Ansible fonctionne normalement via SSH sur des serveurs distants. Ici, toute la stack tourne sur la même machine locale. La directive **ansible_connection=local** dit à Ansible de ne pas ouvrir de connexion SSH mais d'exécuter les commandes directement en local. La directive **ansible_become=yes** autorise l'usage de sudo pour les tâches qui nécessitent des droits d'administration (installation de paquets, modification de fichiers système).

```
[local]

localhost ansible_connection=local ansible_user=lms ansible_become=yes


[harbor]

localhost ansible_connection=local


[jenkins]

localhost ansible_connection=local
```

3.2 Les variables centralisées — `ansible/group_vars/all.yml`

Ce fichier est le **point de vérité unique** de la configuration. Il est automatiquement chargé par Ansible pour tous les playbooks, sans avoir besoin de l'importer explicitement. Si l'on veut changer le mot de passe Harbor, le nombre de replicas Kubernetes, ou l'adresse de la registry, on ne modifie qu'ici — le changement se propage automatiquement à tous les rôles et playbooks qui utilisent ces variables.

```
harbor_hostname: "harbor.local"

harbor_admin_password: "Harbor12345"

harbor_version: "v2.11.0"
```

```
harbor_install_dir: "/opt/harbor"

app_namespace: "production"

app_replicas: 2

# Ces variables sont utilisées automatiquement par tous
# les rôles et playbooks Ansible du projet
```

3.3 Le playbook d'installation — ansible/setup.yml

Ce playbook est lancé **une seule fois** au démarrage du projet pour tout installer. Il appelle les rôles dans l'ordre logique : d'abord Harbor (la registry doit exister avant que Jenkins puisse y pousser des images), puis Minikube (le cluster Kubernetes), puis Jenkins (qui a besoin des deux précédents pour fonctionner). Les **tags** permettent de relancer seulement une partie si quelque chose échoue — par exemple **--tags harbor** pour réinstaller uniquement Harbor.

```
- name: "Setup Harbor Registry"

hosts: local

become: yes # sudo requis pour installer des paquets

roles:

- role: harbor


- name: "Configurer Minikube"

hosts: local

become: no # pas besoin de sudo pour minikube

roles:

- role: minikube


- name: "Configurer Jenkins"

hosts: local
```

```
become: no

roles:

- role: jenkins
```

3.4 Le playbook de déploiement — ansible/deploy.yml

Ce playbook est appelé automatiquement par Jenkins à **chaque build**. Il reçoit le numéro du build comme variable (**image_tag**) et effectue deux opérations critiques.

La première est la création du secret Harbor dans Kubernetes. Ce secret contient les identifiants nécessaires pour que les pods Kubernetes puissent s'authentifier auprès de Harbor et télécharger les images. Sans lui, tous les pods resteraient bloqués en état **ImagePullBackOff**. La logique `get || create` garantit l'idempotence : si le secret existe déjà, il n'est pas recréé.

```
# Création idempotente du secret Harbor

kubectl get secret harbor-registry-secret ... || \

kubectl create secret docker-registry harbor-registry-secret \

--docker-server=harbor.local \

--docker-username=admin \

--docker-password={{ harbor_admin_password }}
```

La seconde opération est la mise à jour de l'image déployée. La commande **kubectl set image** met à jour l'image sans recréer tout le déploiement, ce qui déclenche un **rolling update** : Kubernetes démarre les nouveaux pods avant de terminer les anciens, garantissant ainsi un déploiement sans interruption de service (zéro *downtime*).

```
# Mise à jour de l'image — déclenche un rolling update automatique

kubectl set image deployment/flask-app \

flask-app=harbor.local/cicd/flask-cicd-app:{{ image_tag }}
```

3.5 Le playbook de maintenance — ansible/refresh-kubeconfig.yml

Après chaque redémarrage de Minikube, les certificats TLS du cluster Kubernetes changent. Le kubeconfig (fichier de configuration kubectl) devient alors invalide et Jenkins ne peut plus communiquer avec le cluster. Ce playbook régénère et synchronise automatiquement le kubeconfig en une seule commande :

```
ansible-playbook -i ansible/inventory/hosts.ini ansible/refresh-kubeconfig.yml
```

```
# Ce playbook :  
  
# 1. Vérifie si Minikube tourne, le démarre si besoin  
  
# 2. Régénère le kubeconfig avec --flatten (certificats embarqués en base64)  
  
# 3. Synchronise le fichier pour l'utilisateur jenkins dans le conteneur
```

4 Les rôles Ansible — Harbor, Minikube, Jenkins, Kubernetes

Les rôles Ansible sont des unités de configuration réutilisables. Chaque rôle encapsule toutes les tâches nécessaires pour configurer un composant précis de la stack. Ils sont appelés par les playbooks *setup.yml* et *deploy.yml*.

4.1 Rôle harbor — Installation de Harbor

Harbor est une registry Docker privée avec interface web, scan Trivy intégré et gestion de projets. Ce rôle gère l'installation complète en plusieurs étapes.

- **Certificats TLS auto-signés** : Harbor nécessite HTTPS. Ansible génère une CA (autorité de certification), puis un certificat serveur signé par cette CA avec openssl. Ces certificats sont valides 10 ans.
- **Déploiement via le script officiel** : le script *install.sh --with-trivy* de Harbor déploie automatiquement une stack Docker Compose composée de 10 conteneurs (core, portal, registry, database, redis, jobservice, etc.).
- **Propagation du certificat CA** : pour que Docker accepte de pousser vers harbor.local en HTTPS, le certificat CA doit être copié dans */etc/docker/certs.d/harbor.local/ca.crt* sur l'hôte, dans le conteneur Jenkins, et dans le nœud Minikube (pour que les pods puissent puller les images).
- **Création du projet "cicd"** : Ansible appelle l'API REST de Harbor pour créer le projet qui contiendra toutes les images de l'application.

```
# Création du projet via l'API REST Harbor

- name: "Créer le projet cicd"

  uri:

    url: "https://harbor.local/api/v2.0/projects"

    method: POST

    body: { project_name: "cicd", public: false }
```

4.2 Rôle minikube — Configuration du cluster Kubernetes

Ce rôle gère toute la liaison entre Minikube, Jenkins et Harbor. Il résout notamment un problème réseau fondamental de cette architecture.

Problème réseau : par défaut, Jenkins tourne dans le réseau Docker bridge (172.17.0.0/24) tandis que Minikube tourne dans son propre réseau Docker (192.168.49.0/24). Ces deux réseaux sont isolés. La solution est de connecter Jenkins au réseau Minikube. La tâche est rendue idempotente via **failed_when** — si Jenkins est déjà connecté, l'erreur 'already exists' est ignorée.

```
- name: "Connecter Jenkins au réseau Minikube"

  command: docker network connect minikube jenkins
```

```

register: network_connect

failed_when:

- network_connect.rc != 0

# Idempotent : si déjà connecté, l'erreur est ignorée

- "'already exists' not in network_connect.stderr"

```

Kubeconfig autonome : le fichier kubeconfig référence normalement des certificats par leur chemin sur la machine hôte. Ces chemins n'existent pas dans Jenkins. La solution est d'utiliser **--flatten --minify** pour embarquer les certificats encodés en base64 directement dans le fichier kubeconfig, le rendant totalement autonome.

```

# --flatten embarque les certificats en base64 dans le fichier

# au lieu de pointer vers des chemins qui n'existent pas dans Jenkins

kubectl config view --flatten --minify > /tmp/kube-flat.yaml

cp /tmp/kube-flat.yaml ~/.kube/config

```

4.3 Rôle jenkins — Installation des outils

Ce rôle installe tous les outils nécessaires **dans le conteneur Jenkins** via des commandes *docker exec*. La logique **command || install** garantit l'idempotence : si l'outil est déjà présent, on ne le réinstalle pas.

```

# Exemple : installation idempotente d'Ansible dans Jenkins

- name: "Installer Ansible dans Jenkins"

command: >

docker exec -u root jenkins bash -c

"ansible --version >/dev/null 2>&1 ||

(apt-get update -qq && apt-get install -y ansible -qq)"

# Le rôle installe ainsi : Docker, Python/Bandit,

# Trivy, Gitleaks, Ansible, et configure les certificats Harbor

```

4.4 Rôle kubernetes — Déploiement de l'application

Ce rôle est appelé par *deploy.yml* à chaque build Jenkins. Il gère la création du namespace Kubernetes, la création du secret d'authentification Harbor, l'application du manifest *deployment.yaml*, et le rolling update de

l'image. Il attend la fin du rollout avant de rendre la main à Jenkins.

5 Harbor Registry — La registry Docker privée

Harbor est un registre de conteneurs open-source certifié CNCF (Cloud Native Computing Foundation). Il remplace Docker Hub dans ce projet pour des raisons à la fois économiques, de sécurité et de disponibilité.

5.1 Pourquoi Harbor plutôt que Docker Hub ?

Critère	Docker Hub	Harbor (local)
Hébergement	Cloud public — internet requis	Local sur la machine, hors-ligne possible
Coût	Payant au-delà du quota gratuit	Gratuit, open-source (Apache 2.0)
Scan CVE	Payant (plan Advanced)	Trivy intégré nativement — gratuit
Contrôle d'accès	Tokens API simples	RBAC complet (projets, utilisateurs, rôles)
Réseau Jenkins	Push via internet	Push direct sur le réseau Docker interne
Disponibilité	Dépend de l'infrastructure Docker Hub	Dépend uniquement de la machine locale

5.2 Format des images dans Harbor

Dans Harbor, chaque image est identifiée par un chemin hiérarchique : **registry / projet / nom-image : tag**. Le projet *cicd* est créé automatiquement par Ansible lors de l'installation.

```
# Format des images dans ce projet :  
  
harbor.local / cicd / flask-cicd-app : 42  
  
# registry projet nom image tag = BUILD_NUMBER Jenkins  
  
# Exemples après plusieurs builds :  
  
harbor.local/cicd/flask-cicd-app:1 # premier build  
  
harbor.local/cicd/flask-cicd-app:8 # build #8 – le build final réussi  
  
harbor.local/cicd/flask-cicd-app:latest # toujours la dernière version
```

6 Les outils de sécurité — Trivy, Bandit, Gitleaks

L'approche **DevSecOps** consiste à intégrer la sécurité dans chaque étape du pipeline plutôt que de la traiter comme une phase finale. Trois outils complémentaires couvrent des périmètres différents : le code source, les secrets dans Git, et les vulnérabilités dans les images Docker.

Outil	Ce qu'il analyse	Comment il bloque	Version
Trivy	Images Docker — CVE dans OS et dépendances Python	--exit-code 1 si CRITICAL/HIGH détectée	v0.69.1
Bandit	Code source Python — vulnérabilités et mauvaises pratiques	Code de retour != 0 si severity >= medium	v1.9.3
Gitleaks	Historique Git — tokens, passwords, clés API exposés	--exit-code 1 si secret trouvé	v8.18.4

6.1 Trivy — Double scan CVE

Trivy est appelé à **deux reprises** dans le pipeline, pour deux raisons différentes. Le premier scan (étape 4) analyse l'image construite localement et bloque le pipeline avec **--exit-code 1** si une CVE CRITICAL ou HIGH est détectée — empêchant ainsi de publier une image vulnérable dans Harbor.

Le second scan (étape 7) analyse l'image directement depuis Harbor avec **--exit-code 0**. Il est uniquement informatif : il confirme dans les logs que ce qui est stocké dans la registry est propre, sans bloquer le pipeline. Cette double vérification garantit l'intégrité de bout en bout.

Note importante : le passage de l'image de base Debian (*python:3.12-slim*) vers Alpine (*python:3.12-alpine*) a été imposé par le pipeline lui-même. Debian contenait la CVE-2026-0861 dans libc6, sans correctif disponible. Alpine utilise musl libc, non affectée par cette CVE. Résultat : **0 CVE CRITICAL/HIGH** et une image deux fois plus légère (54.7 MB vs 125 MB).

6.2 Bandit — Analyse statique du code Python

Bandit est un outil SAST (Static Application Security Testing) spécialisé Python. Il analyse le code source sans l'exécuter, cherchant des patterns connus de vulnérabilités. L'option **--severity-level medium** filtre les problèmes mineurs pour se concentrer sur les risques réels. Bandit détecte par exemple les mots de passe hardcodés (B106), l'utilisation dangereuse de pickle (B301), ou les connexions SSL avec *verify=False* (B501).

6.3 Gitleaks — Détection de secrets dans Git

Gitleaks analyse l'**intégralité de l'historique Git** — pas seulement les fichiers courants. Un secret supprimé dans un commit ultérieur reste visible dans l'historique et constitue une fuite réelle. Gitleaks détecte les clés AWS, les tokens GitHub, les mots de passe de bases de données, les certificats privés et des centaines d'autres patterns de secrets.

7 Le déploiement Kubernetes — Manifest et rolling update

Kubernetes (via Minikube) orchestre les conteneurs Flask en production. Le déploiement est géré de façon **déclarative** via un fichier manifest YAML : on décrit l'état désiré du système (2 pods, telle image, tel service) et Kubernetes s'occupe d'y parvenir et de maintenir cet état en permanence.

7.1 Le manifest k8s/deployment.yaml — Deux changements clés

Par rapport à l'architecture précédente utilisant Docker Hub, deux modifications fondamentales ont été apportées dans le manifest Kubernetes :

1. imagePullSecrets : Kubernetes doit s'authentifier auprès de Harbor pour télécharger les images privées. Ce champ référence le secret créé par Ansible. Sans lui, les pods restent bloqués en **ImagePullBackOff**.

```
imagePullSecrets:

- name: harbor-registry-secret # référence le secret créé par Ansible
```

2. Image depuis Harbor : l'URL de l'image pointe maintenant vers la registry locale Harbor au lieu de Docker Hub.

```
# Avant (Docker Hub public) :

image: sylvain849/flask-cicd-app:latest

# Après (Harbor local) :

image: harbor.local/cicd/flask-cicd-app:latest
```

7.2 Rolling update — Déploiement sans interruption de service

La commande **kubectl set image** met à jour l'image sans recréer tout le déploiement, ce qui déclenche un **rolling update**. Kubernetes remplace progressivement les anciens pods par des nouveaux : il démarre un nouveau pod, attend qu'il soit prêt (health check réussi), puis termine un ancien pod — et ainsi de suite jusqu'à ce que tous les pods soient mis à jour. À aucun moment il n'y a moins d'un pod en service : c'est le **zéro downtime**.

✓ **Résultat final** : Après le build #8 (le build de référence de ce projet), le résultat observé est : 2 pods Running dans le namespace production, image harbor.local/cicd/flask-cicd-app:8, service NodePort accessible sur http://192.168.49.2:30080.

8 Flux d'exécution complet — Du git push aux pods Running

Cette section récapitule l'enchaînement complet des événements, depuis l'action du développeur jusqu'au déploiement final en production.

#	Action	Outil	Résultat attendu
1	git push origin main	Git	Code envoyé sur GitHub (branch main)
2	WebHook HTTP POST vers Jenkins	GitHub	Pipeline Jenkins déclenché automatiquement
3	checkout scm	Jenkins	Code source cloné dans Jenkins
4	bandit -r app/ --severity-level medium	Bandit	Aucune vulnérabilité Python détectée
5	docker build --no-cache -t harbor.local/.../flask:N .	Docker	Image Flask Alpine 54.7 MB créée
6	trivy image --exit-code 1 flask-cicd-app:N	Trivy	0 CVE CRITICAL/HIGH sur Alpine
7	gitleaks detect --exit-code 1	Gitleaks	Aucun secret exposé dans Git
8	docker login harbor.local --password-stdin	Docker/Harbor	Login Succeeded
9	docker push harbor.local/cicd/flask-cicd-app:N + :latest	Docker	Image stockée dans Harbor
10	trivy image --exit-code 0 harbor.local/.../flask:N	Trivy	Confirmation 0 CVE (informatif)
11	ansible-playbook deploy.yml -e "image_tag=N"	Ansible	Déploiement K8s déclenché
12	kubectl get secret kubectl create secret ...	kubectl	Secret Harbor présent dans K8s
13	kubectl apply -f k8s/deployment.yaml	kubectl	Manifest appliqué
14	kubectl set image deployment/flask-app flask-app=...:N	kubectl	Rolling update démarré
15	kubectl rollout status deployment/flask-app	kubectl	"successfully rolled out"
16	kubectl get pods -n production	Jenkins	flask-app-xxx 1/1 Running ×2

8.1 Pourquoi Ansible plutôt que kubectl direct ?

Ce tableau résume les bénéfices concrets apportés par Ansible par rapport à l'exécution directe de commandes kubectl dans Jenkins.

kubectl direct	Via Ansible
Commandes manuelles, pas reproductibles	Playbook versionné dans Git (IaC)
Pas de gestion des erreurs	failed_when, retries, until
Variables répétées partout	group_vars/all.yml centralisé
Un seul environnement possible	Inventory multi-env (dev/prod/staging)
Idempotence non garantie	Tâches idempotentes nativement
Difficile à auditer	Playbooks lisibles et versionnés dans Git

★ **En résumé :** Ce pipeline CI/CD constitue une implémentation complète des pratiques DevSecOps : automatisation totale, sécurité intégrée à chaque étape (Bandit + Trivy + Gitleaks), infrastructure as code avec Ansible, registry privée avec Harbor, et déploiement Kubernetes sans interruption de service. L'ensemble est reproductible, traçable et extensible.